

Type Analysis (2)

CSE552 Program Analysis — Lecture 4

Jaemin Hong

Solving Constraints — Mechanisms

- ▶ We use two mechanisms to express equivalences:
 - ▶ Equivalence between type variables is expressed using union-find
 - ▶ Equivalence between a type variable and a proper type is expressed using a mapping M from type variables to types (initially undefined for all)
- ▶ Assume that $\text{MakeSet}(X)$ for each type variable X that occurs in the constraints is called in advance

Solving Constraints — Resolve

- $\text{Resolve}(T)$: resolves a type to its representative

Resolve(T):

```
  if  $T$  is a type variable  $X$  :  
    |  $X^r \leftarrow \text{Find}(X)$   
    | if  $M[X^r]$  is defined :  
    | | return  $M[X^r]$   
    | else  
    | | return  $X^r$   
  else  
    | return  $T$ 
```

Solving Constraints — Unify Algorithm (Part 1)

- For each constraint $T_1 = T_2$, we invoke $\text{Unify}(T_1, T_2)$

Unify(T_1, T_2):

$T_1^r \leftarrow \text{Resolve}(T_1)$

$T_2^r \leftarrow \text{Resolve}(T_2)$

if $T_1^r = T_2^r$:

return

if *both* T_1^r *and* T_2^r *are type variables* :

$\text{Union}(T_1^r, T_2^r)$

else if T_1^r *is a type variable* :

if $\text{Occurs}(T_1^r, T_2^r)$:

 unification fails

else

$M[T_1^r] \leftarrow T_2^r$

Solving Constraints — Occurs Check

- ▶ Occurs check: checks if a type variable X occurs in a type T
 - ▶ Prevents infinite types (e.g., $X = \text{fn}(\text{i32}) \rightarrow X$)

Occurs(X, T):

```
if  $T$  is a proper type  $C(T_1, \dots, T_n)$  :  
    return  $\text{Occurs}(X, T_1) \vee \dots \vee \text{Occurs}(X, T_n)$   
if  $T$  is a type variable  $Y$  :  
     $Y^r \leftarrow \text{Find}(Y)$   
    if  $M[Y^r]$  is defined :  
        return  $\text{Occurs}(X, M[Y^r])$   
    else  
        return  $Y^r = X$ 
```

Solving Constraints — Unify Algorithm (Part 2)

Continuing Unify:

```
if ... (previous cases) :  
  else if  $T_2^r$  is a type variable :  
    | if  $Occurs(T_2^r, T_1^r)$  :  
    |   unification fails  
    | else  
    |    $M[T_2^r] \leftarrow T_1^r$   
  else if  $T_1^r = C(T_1', \dots, T_n')$  and  $T_2^r = C(T_1'', \dots, T_n'')$  :  
    | for  $i$  in  $1..n$  :  
    |   Unify( $T_i', T_i''$ )  
  else  
    | unification fails
```

Solving Constraints — Obtaining the Solution

- Once Unify is called for each constraint, calling DeepResolve on each type variable gives the solution

DeepResolve(T):

```
if  $T$  is a proper type  $C(T_1, \dots, T_n)$  :  
    return  $C(\text{DeepResolve}(T_1), \dots, \text{DeepResolve}(T_n))$   
if  $T$  is a type variable  $X$  :  
     $X^r \leftarrow \text{Find}(X)$   
    if  $M[X^r]$  is defined :  
        return  $\text{DeepResolve}(M[X^r])$   
    else  
        return  $X^r$ 
```

Solving Constraints — Implementation

- ▶ Each constraint is processed only once
- ▶ For implementation, we can interleave the collection and solving phases, solving the constraints on-the-fly, as they are being generated

Tuple Types

Expression $e ::= \dots \mid (e, \dots, e) \mid e.i$

Type $T ::= \dots \mid (T, \dots, T)$

- ▶ We want to extend the type analysis to support tuple types
 - ▶ Projection on non-tuple types should be rejected
 - ▶ Projection on non-existent elements of tuples should be rejected

Tuple Type Constraints (First Attempt)

- ▶ $(e_1, \dots, e_n): \llbracket (e_1, \dots, e_n) \rrbracket = (\llbracket e_1 \rrbracket, \dots, \llbracket e_n \rrbracket)$
- ▶ $e.i: \llbracket e \rrbracket = (X_0, \dots, X_{i-1}, \llbracket e.i \rrbracket)$
 - ▶ where $X_0, \dots, X_{i-1}$ are fresh type variables

Tuple Type Constraints (First Attempt) — Example

```
fn f() {  
  let x = (1, true);  
  x.0  
}
```

Constraints:

- ▶ $\text{fn } f() \{ \dots \}: \llbracket f \rrbracket = \text{fn}() \rightarrow \llbracket x.0 \rrbracket$
- ▶ $\text{let } x = (1, \text{true}): \llbracket x \rrbracket = (\llbracket 1 \rrbracket, \llbracket \text{true} \rrbracket)$
- ▶ $1: \llbracket 1 \rrbracket = \text{i32}$
- ▶ $\text{true}: \llbracket \text{true} \rrbracket = \text{bool}$
- ▶ $x.0: \llbracket x \rrbracket = (\llbracket x.0 \rrbracket)$

No solution exists because $\llbracket x \rrbracket$ cannot be both $(\llbracket x.0 \rrbracket)$ and $(\text{i32}, \text{bool})$.

Tuple Type Constraints (Second Attempt)

Let N be the largest tuple length or projection index in the program.

- ▶ $(e_1, \dots, e_n)$: $\llbracket (e_1, \dots, e_n) \rrbracket = (\llbracket e_1 \rrbracket, \dots, \llbracket e_n \rrbracket, X_{n+1}, \dots, X_N)$
- ▶ $e.i$: $\llbracket e \rrbracket = (X_0, \dots, X_{i-1}, \llbracket e.i \rrbracket, X_{i+1}, \dots, X_{N-1})$

Tuple Type Constraints (Second Attempt) — Example

```
fn f() {  
  let x = (1, true);  
  x.2  
}
```

Constraints:

- ▶ $\text{fn } f() \{ \dots \}: \llbracket f \rrbracket = \text{fn}() \rightarrow \llbracket x.2 \rrbracket$
- ▶ $\text{let } x = (1, \text{true}): \llbracket x \rrbracket = (\llbracket 1 \rrbracket, \llbracket \text{true} \rrbracket, X_3)$
- ▶ $1: \llbracket 1 \rrbracket = \text{i32}$
- ▶ $\text{true}: \llbracket \text{true} \rrbracket = \text{bool}$
- ▶ $x.2: \llbracket x \rrbracket = (X_1, X_2, \llbracket x.2 \rrbracket)$

Solution: $\llbracket x \rrbracket = (\text{i32}, \text{bool}, X_3), \quad \llbracket f \rrbracket = \text{fn}() \rightarrow X_3$

Tuple Type Constraints (Correct)

- ▶ Add a type to represent an absent element of a tuple

$$T ::= \dots \mid \diamond$$

- ▶ $(e_1, \dots, e_n): \llbracket (e_1, \dots, e_n) \rrbracket = (\llbracket e_1 \rrbracket, \dots, \llbracket e_n \rrbracket, \diamond, \dots, \diamond)$
- ▶ $e.i: \llbracket e \rrbracket = (X_0, \dots, X_{i-1}, \llbracket e.i \rrbracket, X_{i+1}, \dots, X_{N-1}) \wedge \llbracket e.i \rrbracket \neq \diamond$

Tuple Type Constraints (Correct) — Example 1

```
fn f() {  
  let x = (1, true);  
  x.0  
}
```

Constraints:

- ▶ $\text{fn } f() \{ \dots \}: \llbracket f \rrbracket = \text{fn}() \rightarrow \llbracket x.0 \rrbracket$
- ▶ $\text{let } x = (1, \text{true}): \llbracket x \rrbracket = (\llbracket 1 \rrbracket, \llbracket \text{true} \rrbracket)$
- ▶ $1: \llbracket 1 \rrbracket = \text{i32}$
- ▶ $\text{true}: \llbracket \text{true} \rrbracket = \text{bool}$
- ▶ $x.0: \llbracket x \rrbracket = (\llbracket x.0 \rrbracket, T_2)$

Solution: $\llbracket x \rrbracket = (\text{i32}, \text{bool})$, $\llbracket f \rrbracket = \text{fn}() \rightarrow \text{i32}$

Tuple Type Constraints (Correct) — Example 2

```
fn f() {  
  let x = (1, true);  
  x.2  
}
```

Constraints:

- ▶ $\text{fn } f() \{ \dots \}: \llbracket f \rrbracket = \text{fn}() \rightarrow \llbracket x.2 \rrbracket$
- ▶ $\text{let } x = (1, \text{true}): \llbracket x \rrbracket = (\llbracket 1 \rrbracket, \llbracket \text{true} \rrbracket, \diamond)$
- ▶ $1: \llbracket 1 \rrbracket = \text{i32}$
- ▶ $\text{true}: \llbracket \text{true} \rrbracket = \text{bool}$
- ▶ $x.2: \llbracket x \rrbracket = (X_1, X_2, \llbracket x.2 \rrbracket) \wedge \llbracket x.2 \rrbracket \neq \diamond$

No solution exists because $\llbracket x.2 \rrbracket$ should not be $\diamond$.

Tuple Type Constraints — Implementation

- ▶ Unify is for solving equalities
- ▶ We first apply Unify to solve all the equalities, and then check the inequalities
 - ▶ If any inequality is violated, the analysis says “not ok”
 - ▶ Otherwise, the analysis says “ok”

Limitations — Flow-Insensitivity

```
fn f() {  
  let x = 1;  
  let y = x + 2;  
  x = true;  
  if x { y } else { 0 }  
}
```

- ▶ It does not incur a type error at runtime
 - ▶ Common pattern in dynamically typed languages
- ▶ However, the analysis will say “not ok”
- ▶ The analysis ignores the order of execution and computes a single type for each identifier regardless of the program point at which it is used
- ▶ Such an analysis is called a flow-insensitive analysis
- ▶ We will cover flow-sensitive analyses later in the course

Limitations — Polymorphism (Problem)

```
fn f(x) { x }  
fn g() {  
    f(1);  
    f(true)  
}
```

- ▶ It does not incur a type error at runtime
 - ▶ `f` is polymorphic (generic function)
 - ▶ We can type it
`fn f<T>(x: T) -> T { x }` in Rust
- ▶ However, the analysis will say “not ok”
 - ▶ $\llbracket x \rrbracket$ needs to be both `i32` and `bool`

Limitations — Polymorphism (Solution)

- ▶ To address this, we need to instantiate a function with different types at different call sites
 - ▶ $\llbracket f \rrbracket = \forall X. \text{fn}(X) \rightarrow X$
 - ▶ In $f(1)$, $\llbracket f \rrbracket$ is instantiated to $\text{fn}(i32) \rightarrow i32$
 - ▶ In $f(\text{true})$, $\llbracket f \rrbracket$ is instantiated to $\text{fn}(\text{bool}) \rightarrow \text{bool}$
- ▶ This is the key idea of Hindley-Milner algorithm and often called let-polymorphism
- ▶ The time complexity is exponential in the worst case¹²
- ▶ Later in the course, we will cover context-sensitive analyses, which distinguish different call sites of a function and are similar to let-polymorphism

¹Deciding ML typability is complete for deterministic exponential time (Mairson, 1990)

²ML typability is Dextime-complete (Kfoury et al., 1990)

Let-Polymorphism

- ▶ Even with let polymorphism, false alarms are unavoidable
- ▶ No higher-rank polymorphism

```
fn f(x) { x }  
fn g(y) {  
  y(1);  
  y(true)  
}  
fn h() { g(f) }
```

- ▶ `y` is a parameter and cannot be polymorphically instantiated at different call sites

Let-Polymorphism (cont.)

- ▶ No polymorphic recursion

```
fn f(x, n) {  
  if n > 1 {  
    f(true, n - 1);  
  } else if n == 1 {  
    f(0, n - 1);  
  }  
  x  
}
```

- ▶ Each recursive call requires x to have a different type
- ▶ With unrestricted higher-rank polymorphism or polymorphic recursion, solving the constraints becomes undecidable³⁴⁵

³Type inference with polymorphic recursion (Henglein, 1993)

⁴Type reconstruction in the presence of polymorphic recursion (Kfoury et al., 1993)

⁵Principality and decidable type inference for finite-rank intersection types (Kfoury and Wells, 1999)

Limitations — Other Runtime Errors

- ▶ Division by zero
- ▶ Stack-use-after-scope
 - ▶ Catching dangling references requires lifetime/region analysis, which is beyond the scope of this type analysis

```
fn f() {  
    let x = 0;  
    &x  
}  
fn g() {  
    *f()  
}
```

Summary

- ▶ Constraints are solved by the Unify algorithm, which uses union-find for type variable equivalences and a mapping for variable-to-type bindings
- ▶ Tuple types require an absent-element type $\diamond$ and inequality constraints to correctly reject out-of-bounds projections
- ▶ The analysis is flow-insensitive and monomorphic; let-polymorphism addresses the latter but with exponential worst-case complexity